

DEPARTAMENTO DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Trabajo práctico 2: Implementacion AirMiles

Tad AirMiles

November 8, 2026 — Algoritmos y Estructuras de Datos

Grupo: Arraydecuatroposiciones

Integrante	LU	Correo electrónico
Javier M. Ramos	288/20	ramosjavier276@gmail.com
Nancy Ramos	1206/24	Ramosnancydaniela@outlook.com
Elian Carreño	1420/23	elianuba2004@gmail.com
Leandro Amodey	618/24	leandroamodey@gmail.com

Facultad de Ciencias Exactas y Naturales

Universidad de Buenos Aires

Ciudad Universitaria - (Pabellón I/Planta Baja)

Intendente Güiraldes 2610 - C1428EGA

Ciudad Autónoma de Buenos Aires - Rep. Argentina

Tel/Fax: (++54 +11) 4576-3300

http://www.exactas.uba.ar

1 Trabajo Practico 2 - Justificaciones

Para la segunda parte de la materia, implementamos en codigo algunas de las funcionalidades descriptas en la especificacion de AirMiles.

Para esta implementacion, en primer lugar elejimos las estructuras de datos que cumplen con las complejidades pedidas. En esta eleccion tuvimos que decidir de que manera agrupar los datos primero (Usuarios y Promociones) y luego de que manera almacenarlos en memoria (Heap, ABB o Trie).

A continuacion daremos una exposicion de como lo pensamos inicialmente y luego como es que cada procedimiento/funcion cumple con las complejidades pedidas.

1.1 Clases

Elegimos tener 2 clases que representen nuestros objetos: Usuarios y Promociones.

En Usuarios, guardamos la informacion de millas disponibles y totales, categoria actual y datos necesarios para conocer a los transferidores exclusivos.

En Promociones, guardamos el nombre de la misma, y los datos para saber que factor aplicar en funcion de si tiene tope o no.

1.2 Estructuras

Pasando a las estructuras elegidas, utilizamos 3: una Arbol Binario de Busqueda, un Heap y un Trie.

El ABB, lo utilizamos como estructura principal para guardar y recuperar Usuarios. De esta manera, en el caso promedio, que es que los IDs de los usuarios esten distribuidos de manera aleatoria, tendriamos un orden de complejidad O(log U). En el peor caso que seria siempre insertar IDs en orden, el orden de complejidad es similar al de una lista enlazada que es O(U)

El Trie, lo utilizamos como estructura para almacenar, recuperar y eliminar las promociones a partir del nombre de la misma. Esto nos permite incorporar la complejidad O(tamaño(L)) que es requerida en alguno de los procedimientos.

Por ultimo el MaxHeap, lo utilizamos como estructura paralela para los usuarios, ya que de esta manera podremos ordenarlos de acuerdo a un comparador, y obtenerlos en tiempos mas acotados a que si hubiera que hacerlo mediante un algoritmo. Esta estructura esta vinculada con los usuarios mediante Handles, de manera que los tengamos como una referencia directa y no haya que realizar una doble busqueda.

1.3 Complejidad de los procedimientos

1.3.1 Crear Sistema

No requiere demasiada explicacion. Inicializar las estructuras, se realiza en un tiempo fijo que no depende de una entrada, por lo cual la complejidad es O(1)

1.3.2 Registrar Usuario - Registrar Referido

En este caso, la operacion mas costosa es la de insertar el usuario en el ABB. Esto como dijimos se realiza en O(log(U)) en promedio y en O(U) en el peor caso.

Tambien actualizamos los Heaps, pero en esa estructura la complejidad es O(log(U)) en el peor caso, y se suma, por lo cual no cambia lo anterior.

1.3.3 Acumular Millas

Aca la busqueda de la Promocion se realiza en O($|L|$). En el caso de que haya que eliminarla por el Tope, se suma la misma complejidad.

Por otro lado la busqueda del Usuario, se realiza en O(log(U)) en promedio y en O(U) en el peor caso.

La actualizacion del Heap, tiene la misma complejidad que el caso anterior.

Finalmente, tomando los tiempos mas grandes de cada variable, y teniendo en cuenta que las constantes no suman, nos queda O(log U + $|L|$) en el caso promedio, y O(U + $|L|$) en el peor caso.

1.3.4 Canjear Millas

En este acso, es similar a Registrar Usuario, la complejidad es buscarlo. Luego modificarlo es O(1) y actualizar los Heaps O(log(U)).

1.3.5 Transferir Millas

Idem a Canjear Millas. La unica aclaracion es que aunque se busque 2 veces, al ser una cantidad constante de veces, no afecta a la complejidad.

1.3.6 Consultar Saldo

Idem a Canjear y Transferir Millas, la busqueda del Usuario es lo que lleva la complejidad. Luego devolver el dato es O(1)

1.3.7 Iniciar Promocion

Iniciar promociones se realiza en O($|L|$), pues los indices donde guardar cada letra son fijos, y depende del tamaño del nombre de la promocion

1.3.8 Finalizar Promocion

Para finalizar promociones, se realiza en O($|L|$) 2 veces, por la busqueda y la eliminacion.

1.3.9 Mas Acumulador - Mas Canjeador

Como estamos usando MaxHeaps, en ambos casos la complejidad es O(1), ya que segun el criterio de comparaacion, tendremos al maximo en la posicion 0 del Array.

1.3.10 Ranking Millas

En este caso, tambien usamos el MaxHeap, pero necesitamos desencolar el elemento Maximo. Para desencolar es necesario realizar un ShiftUp o ShiftDown, esto se realiza en O(log(U)) y lo tenemos que hacer N veces para la cantidad de usuarios rankeados, llegando ala complejidad O(N log(U))

1.3.11 Pares Transferidores Exclusivos

Para crear los Pares Transferidores utilizamos un iterador del ABB, creamos un ArrayList para guardar los transferidores exclusivos y luego con el tamaño final lo pasamos a un array bidimensional que es lo que se pide devolver.

Crear el iterador nos lleva O(U) en el peor caso porque debemos encontrar el minimo. Iterar inorder, es O(U) ya que debemos recorrer todos los nodos, y por ultimo pasar el ArrayList al array bidimensional, es O(u) siendo u (minuscula) la cantidad de pares encontrados, y nos queda finalmente O(U) como complejidad total.